data-model

Data Model — Chapter Hierarchy

Feature: `specs/003-chapter-hierarchy` | **Date**: 2026-09-05

Normative dependencies: `specs/001-workshop-curriculum-platform/data-model.md` defines the Chapter entity and `specs/001-workshop-curriculum-platform/contracts/http-api.md` defines its wire form. **This document amends both**, and the amendment is not editorial — see §1.1. Where this document and a 001 contract disagree on anything *other* than the two amendments named there, the 001 contract wins and this document is the defect.

0. What already exists and is not redefined here

Restated only to fix the boundary. None of this is changed by this feature.

- **Passage** — the smallest addressable unit of source content. Carries a minted 26-character identifier, a kind, a **chapter `scope`**, an ordering key, a source reference, text, provenance, confidence, an uncertainty flag, a redaction flag and a change-detection hash. Measured 2026-09-05: **13,141 records**, of which **1,819** carry the chapter scope `01`, **2,176** carry `docs`, **2** carry `curriculum`, and **9,144** carry no scope. **The `scope` field is already an opaque string and already holds a chapter id. This feature changes which strings appear in it, not the field.**
- **Chapter Section** — derived, never authored, with its own derivation rule and cross-check.
- **Cross-reference** — an existing passage-to-passage relation.

- **The knowledge layer** of feature 002 — areas, terms, lesson sections, questions, mentions. All flat, all consuming `chapter_slug`, none of them changed here.

1. Identity — one string, five roles

The canonical chapter identifier is the **dotted, zero-padded numeric path**:

`^[0-9]{2,}(\.[0-9]{2,})*$`

`01` · `02` · `02.01` · `02.01.01` · `100` are valid. `2` · `02.2` · `02.` · `.01` · `02.01.` · the empty string are not.

That one string serves, **unchanged and untranslated**, as all five of:

Role	Where	Measured today
directory name under `chapters/`	the filesystem	`chapters/02.01` exists
URL path segment	`GET /api/chapters/{chapter}` and five siblings	`main.go:547`, `:817–:819`,`:841`,`:843`
passage registry `scope`	`curriculum/passages.jsonl`	`01` present; `02` and `02.01` hold zero records
`chapter_slug` field value	the knowledge layer's artifacts	`chapter-01/knowledge/build.py:415` derives it correctly from `r["scope"]`
suffix of a `chapter-<id>` directory	`curriculum/chapter-01/`	`ChapterDir` (`curriculum.go:187`) tries `chapter-<slug>` then `<slug>`

No layer maps it to a different form. There is no slug-to-ordinal translation, no ordinal-to-directory translation and no display-name lookup. The reason is that every translation is a place where two representations of one chapter can disagree, and the disagreement is silent — the link still resolves, to something else.

1.1 The two inherited contradictions this identity forces into the open

Both are **defects in feature 001's own artifacts**, both pre-date this feature, and both were invisible while there was one chapter — because with `01` alone, the slug and the zero-padded ordinal are the same string.

1. **`specs/001-.../data-model.md:45` types `Chapter.ordinal` as `int`.** `02.01` has no `int` ordinal. `ordinalOf` (`internal/api/chapters.go:736`) is the faithful implementation of that type and it returns **`2`** for both `02` and `02.01`; `chapterTitle` (`:755`) renders both as *"Chapter 2"*. **The collision is the type, not the function.**
2. **`specs/001-.../contracts/http-api.md:60` states that the zero-padded `ordinal` is "not accepted as a path key — one key, one meaning".** The live, manifest-declared route is `GET /api/chapters/01` (`route-manifest.tsv:72`), and `01` **is** the zero-padded ordinal.

This document's proposal is to widen `ordinal` into an ordered **`ordinal_path`** and to amend `http-api.md:60` to say that the dotted id **is** the path key — which makes "one key, one meaning" true for the first time rather than merely stated. **Neither amendment may be made by implementing it.** Both are recorded as the feature's blocking decision (spec §Clarifications), and every field below marked `ordinal_path` is provisional on it.

1.2 What the identity is *not*

A chapter id is not a minted identifier, and this feature does not make it one. The platform's identity contract mints ULIDs for passages, areas, terms, questions and mentions precisely because those are content-bearing entities whose text gets corrected, and a content-derived identifier would break every citation when it is. A chapter id is a **structural name under an operator's control**, chosen deliberately, never generated, and immutable once published — the same status `specs/001-.../data-model.md` already gives `Chapter.slug`.

The consequence is stated so it is not discovered: **a chapter cannot be re-parented without being renamed**, because its parent is derived from its name (§3). That is accepted. Re-parenting a chapter changes what it *is*, and a rename is a visible operation with a visible diff, where editing a stored parent field is neither.

2. Entities

2.1 Chapter

Field	Type	Invariant
`id`	string	matches §1's grammar; **immutable once published**; unique. It is the directory name, the path key, the `scope` and the `chapter_slug`
`ordinal_path`	ordered list of integers	**derived** from `id` by splitting on `.`; provisional on the §1.1 decision. Replaces `ordinal`
`title`	string	**must not collide** between two distinct chapters (FR-023)
`summary`	string or null	unchanged from 001 — `null`, never a placeholder, because no summary is authored and nothing local can write one
`status`	enum	unchanged from 001: `draft` | `transcribed` | `published`
`recording`, `materials`, `transcript`	unchanged	a **parent chapter may legitimately have none of them** — see below
`hierarchy`	derived view	§2.2. **Never stored**

A parent chapter with no material of its own is a valid chapter, not an error. `02` may be a container for `02.01`. Its zero passages are a measurement to publish, not a condition to raise. Nothing in the model distinguishes a container from a leaf, and nothing should — that distinction is `child_ids` being non-empty, which is derived.

`id` is immutable once published for the same reason `slug` already is: every deep link, every registry `scope` value, every archived artifact and every `chapter-<id>` directory resolves through it.

2.2 Chapter Hierarchy — a derived view, not an entity

Named here only because the API returns it, and a thing the API returns needs a defined shape. **It has no storage, no table, no column and no file.**

Field	Derivation from `id` alone	Root / leaf behaviour
`parent_id`	everything before the last `.`; `null` if there is no `.`	`null` for a top-level chapter — **falls out**, not special-cased
`depth`	count of components — one plus the number of `.`	`1` for a top-level chapter
`ancestor_ids`	every proper prefix ending at a component boundary, outermost first	`[]` for a top-level chapter
`child_ids`	every present id whose `parent_id` equals this id, in byte order	`[]` for a leaf — **falls out**, not special-cased
`ordinal_path`	each component parsed as an integer, in order	a single-element list for a top-level chapter
`orphaned`	true when `parent_id` is non-null and names an id not present	false for a top-level chapter

Worked, against the ids measured present plus the synthetic cases the gates use:

id	parent_id	depth	ancestor_ids	ordinal_path
01	null	1	[]	[1]
02	null	1	[]	[2]
02.01	02	2	["02"]	[2, 1]
02.01.01	02.01	3	["02", "02.01"]	[2, 1, 1]
02.10	02	2	["02"]	[2, 10]
10	null	1	[]	[10]

`child_ids` is the one field that needs the id *set* rather than the id alone, and `orphaned` is the one that needs it too. Both are still derivations — over the set of present ids, computed on demand — and neither is stored. Naming that distinction matters: it is the seam where somebody will later propose "just cache the children", which is §3's forbidden second representation wearing a performance argument. The measured chapter count is **3**.

3. The derived-not-stored rule

> **`parent_id`, `depth`, `ancestor_ids`, `child_ids` and `ordinal_path` are ALL derived from the identifier by string operation, and NONE of them is stored — so they cannot disagree with it.**

This is the model's central decision and it is one sentence long on purpose.

What it forbids, concretely, because each of these is the reasonable-looking form:

- a `parent_id` column on a chapter table — a chapter whose id says `02.01` and whose stored parent says `03` is **not a detectable error state**. It is two facts, both readable, both plausible, with no symptom until something navigates;
- a `depth` field written at ingest — correct when written, stale after a rename;
- a materialised child list — correct until a chapter is added, and nothing tells it one was;
- a stored `ordinal_path` — the same collision as `ordinal`, one indirection further away;

- a hierarchy sidecar file listing the tree — a second enumeration that agrees today and is what a reader will trust tomorrow.

Why this and not a stored field. The argument for storing is always the same and it is always about cost: the derivation is repeated work. Measured, the work is a string split over **3** ids. The argument against storing is not about cost at all: *two representations of one fact can disagree, and this particular disagreement has no symptom.* A navigation that follows a wrong stored parent does not fail — it succeeds, into the wrong chapter, and every artifact downstream of it agrees with itself.

This is the same reasoning feature 002 applied when it refused stored reverse edges (`002/data-model.md` §3) and the same reasoning feature 001 applied when it kept chapter sections derived. It is recorded here rather than cross-referenced because a reader arriving at a `parent_id` proposal needs the argument in front of them, not a pointer to it.

4. Ordering

Byte-lexicographic on the `id` string. Nothing else.

Verified by execution rather than reasoning, 2026-09-05:

```
printf '10\n02.09\n01\n03\n02\n02.10\n02.01.01\n02.01\n' |
        LC_ALL=C sort
```

Position	1	2	3	4	5	6	7	8
id	01	02	02.01	02.01.01	02.09	02.10	03	10

A parent sorts immediately before its first child, and that is a consequence rather than a rule. `.` is **0x2E**; the digits `0`–`9` are **0x30–0x39**. So at the position where a parent's id ends, a child's `.` compares below any sibling's next digit, and `02` precedes `02.01` precedes `03`. Nothing implements this; it happens.

`02.10` after `02.09` is the case that proves it is not a decimal comparison. Read as decimals, `.09` and `.10` would order the same way — but `02.9` (unpadded) would sort *after* `02.10`, which is exactly why the padding is in the grammar. **The ordering is true only because every component is zero-padded, which makes the validator the mechanism that carries it.**

The comparator already exists and is already correct. `cmd/workshop-server/main.go:2126` reads `sort.Slice(out, func(i, j int) bool { return out[i].ID < out[j].ID })`. **It MUST NOT be changed.** The assertion goes on the *ordering* — API order, shell-glob order and filesystem byte order must be the same order (H4) — so that a future rewrite into a "smarter" numeric comparator is caught by the gate rather than by a reader noticing `02.10` in the wrong place.

5. Invariants the model must not be able to violate

Stated as invariants rather than as tests, because a test can be deleted and a structural property cannot.

- **H1 — A directory under `chapters/` that does not match the grammar is reported as `unclassified`, with a reason, in every enumeration that covers it.** It is never silently skipped and never silently accepted. The failure being guarded is not a crash; it is a green run over an incomplete set — which is exactly what `chapter-\d+` plus `continue` produced before `266f443`.
- **H2 — Every hierarchy field is derived; none is stored.** §3. There is no column, no file and no cache holding a parent, a depth or a child list.
- **H3 — An orphaned sub-chapter is still SERVED**, with `orphaned: true` and the missing ancestor named. Hiding it loses content that exists; promoting it to a root silently rewrites the hierarchy. Both alternatives are wrong in a way the reader cannot see, and serving it with a flag is the only form that states what is true.
- **H4 — API order, shell-glob order and filesystem byte order are the same order by construction**, not by coincidence and not by a sorting step that reconciles them. The three are compared element by element and any difference is a failure.
- **H5 — No sub-chapter-specific code path exists anywhere.** A branch conditioned on a chapter's depth — `if depth > 1` — **is a defect**, because it means the general case was not built and the special case will drift away from it. There is one kind of chapter.
- **H6 — Depth is unbounded in the grammar and measured in the response.** The maximum depth present is published as a figure; no constant bounds it. A limit asserted without a measurement is a guess that later gets designed around.
- **H7 — Every chapter-scope comparison is EQUALITY.** `HasSuffix`, `HasPrefix` or `Contains` applied to a chapter scope **is a defect**. This is written as an invariant and not as a note about one file because the

file it was found in — `cmd/workshop-redact/main.go`, now equality at `:616` — is not the only place a scope is compared, and the next one will look just as harmless.

H7 has a corollary that is the whole reason it is an invariant: the defect is invisible to a single-scope fixture. `02.01` has the suffix `01`, so `HasSuffix` and `==` return the same answer for every input a one-chapter test can construct. **A test for H7 must use a fixture registry holding at least two scopes, one a sub-chapter of the other** — and the paired proof must demonstrate that the one-scope fixture stays green under the mutation, because that is the measurement that shows why the cheaper fixture is not coverage.

6. Backward compatibility

Every claim here is a property of the derivation, not a compatibility shim, and each is measurable.

- **`01` is valid under the grammar.** Two digits, no separator. Nothing about the existing corpus becomes invalid.
- **`parent_id: null` and `child_ids: []` are not special cases.** They fall out of §2.2's derivation — no `.` yields no parent; no present id has `01` as a prefix component, so no children. A model that needed a root case would be a model with two rules.
- **`ordinal` keeps its exact current value for every chapter that exists today.** `01` → `[1]`, `02` → `[2]`; read as a single number, both are what `ordinalOf` returns now. The widening changes no existing value; it makes a previously unrepresentable one representable.
- **The two regex changes are WIDENINGS, and their paired mutations must assert BOTH directions.** `chapter-\d+` → `chapter-\d+(?:\.\d+)*` at `meeting_notes.py:109` and `author.py:159`. A proof asserting only that `chapter-02.01` now matches would pass against a pattern that had stopped matching `chapter-01` — which would be a worse defect than the one being fixed, and silent in the same way.
- **The `HasSuffix` → equality change is strictly NARROWING**, and that is why it needs the two-scope fixture. It removes matches; it adds none. On a one-scope registry it is a **no-op** — the two operators agree on every input — so it can be reverted with every existing test still green. The fixture shape is the guard, not the assertion.

7. What this model deliberately does not have

- **No stored hierarchy of any kind.** §3.
- **No chapter-id minting.** §1.2. A chapter id is a structural name, not a content identifier.
- **No display-name table.** `title` is derived from the id, as it already is. Authoring chapter titles is a separate decision and would create a second thing that can be stale.
- **No depth limit.** H6.
- **No `is_sub_chapter` flag, and no chapter kind.** A flag is the storage form of H5's forbidden branch — it exists to be tested against, and the first `if chapter.is_sub` is the special path the invariant forbids. What a caller actually wants is `depth` or `parent_id`, both derived.
- **No re-parenting operation.** §1.2. Re-parenting is a rename, and a rename of a published chapter is not an operation this model offers.
- **No nested representation on the wire.** The `chapters` array stays flat; `api.ts:88` reads it flat, and ancestry is carried per row instead.